Software Design Analysis Report: CareerAI

Session 8 deliverable (Software Design Principles) applying modularization, coupling and cohesion analysis, design approach and design strategy to the proposed CareerAI platform. Items marked "Proposed" are design decisions, not confirmed implementation facts.

A. Project Details

Field	Details
Project title	CareerAI: AI Career Guidance and Skill Recommendation System
Team / roll numbers	Aditi Gupta (202401100100020) Vidhi Goyal (202401100700209) Arushi Gupta (202401100100061)
Project objective	CareerAI is a proposed web-based platform that helps students, fresh graduates and early-career professionals choose careers. It analyses their academic background, interests and skills, recommends careers, identifies skill gaps and suggests learning resources, while letting them track progress.
Technology stack	Responsive web platform with an AI-driven recommendation component; specific languages, frameworks, databases and AI libraries are intentionally left open.

B. Module Identification and Modularization

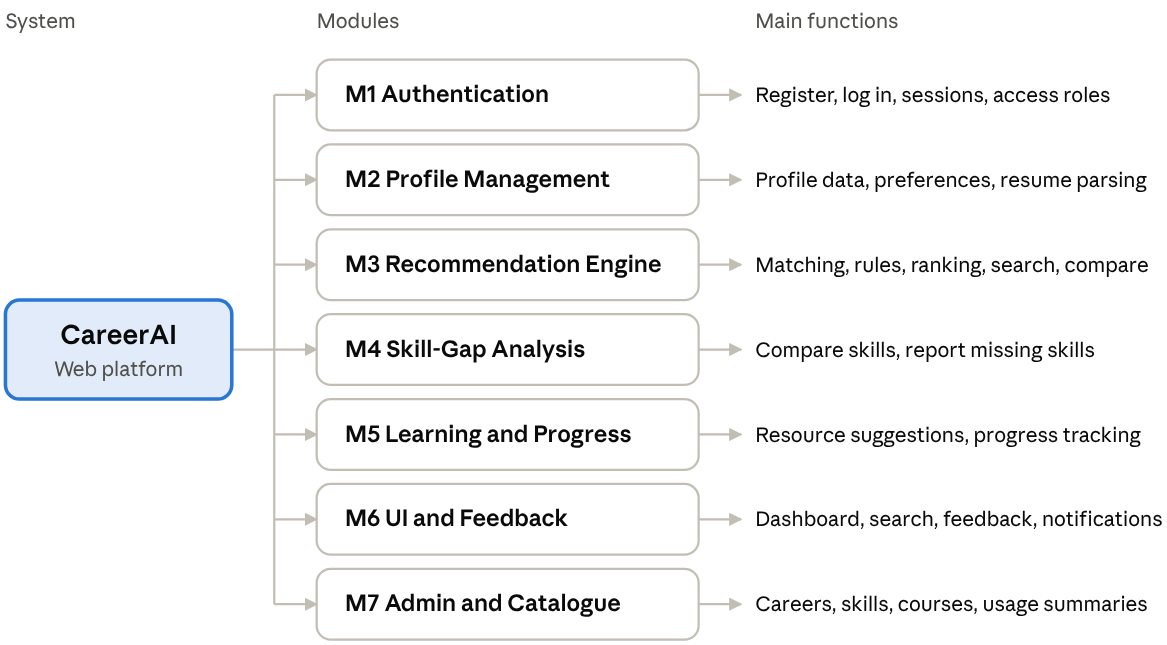
Total number of modules: 7 (M1 to M7). CareerAI is split along its functional responsibilities so that each module can be designed, tested and changed with minimal effect on the others. Security (M1), user data (M2), career intelligence (M3, M4), learning support (M5), presentation (M6) and catalogue maintenance (M7) change for different reasons and at different rates, so they sit in separate modules. The split also keeps the

recommendation and skill-gap logic independent of the interface, which supports maintainability, testing and scalability. All dependencies below are proposed logical dependencies, not a confirmed implementation.

Module ID	Module name	Main responsibility	Inputs / outputs	Depends on
M1	User Registration and Authentication	Register users, log them in securely (email/password or third-party such as Google), manage sessions and access roles.	In: credentials, third-party token. Out: authenticated session, user identity and role.	None (own credential store)
M2	User Profile Management	Create and update profiles holding academic background, interests, skills and preferences; optionally extract skills from an uploaded resume.	In: profile details, optional resume. Out: normalized profile data.	M1
M3	Career Recommendation Engine	Match normalized profiles to career requirements, apply the four recommendation business rules, rank results and support search, filtering and comparison.	In: normalized profile, career and skill data. Out: ranked career recommendations with descriptions.	M2, M7
M4	Skill-Gap Analysis	Compare a user's current skills with the skills required for a selected career and report missing or underdeveloped skills.	In: user skills, selected career requirements. Out: list of skill gaps.	M2, M7 (career selected from M3)

Module ID	Module name	Main responsibility	Inputs / outputs	Depends on
M5	Learning Resource Recommendation and Progress Tracking	Recommend courses, certifications and materials for identified gaps; record in-progress and completed learning activities and show progress.	In: skill gaps, course catalogue, learning activity updates. Out: suggested resources, progress records.	M4, M7
M6	User Interface and Feedback	Responsive dashboard and navigation showing recommendations, gaps, resources and progress; career search and filtering; recommendation feedback; notifications.	In: user actions and feedback. Out: rendered views and requests to the services.	M1, M2, M3, M4, M5
M7	Administration and Career Data Management	Let authorized administrators add, update and remove careers, skills and courses; provide usage summaries and popular careers or skills.	In: catalogue changes, usage data. Out: maintained career, skill and course data; administrative summaries.	M1 (admin role check)

CareerAI splits into seven modules, each with one responsibility



module hierarchy · 1 system, 7 modules

The tree shows the proposed decomposition of CareerAI into seven modules and the main functions each one owns.

C. Design Strategy

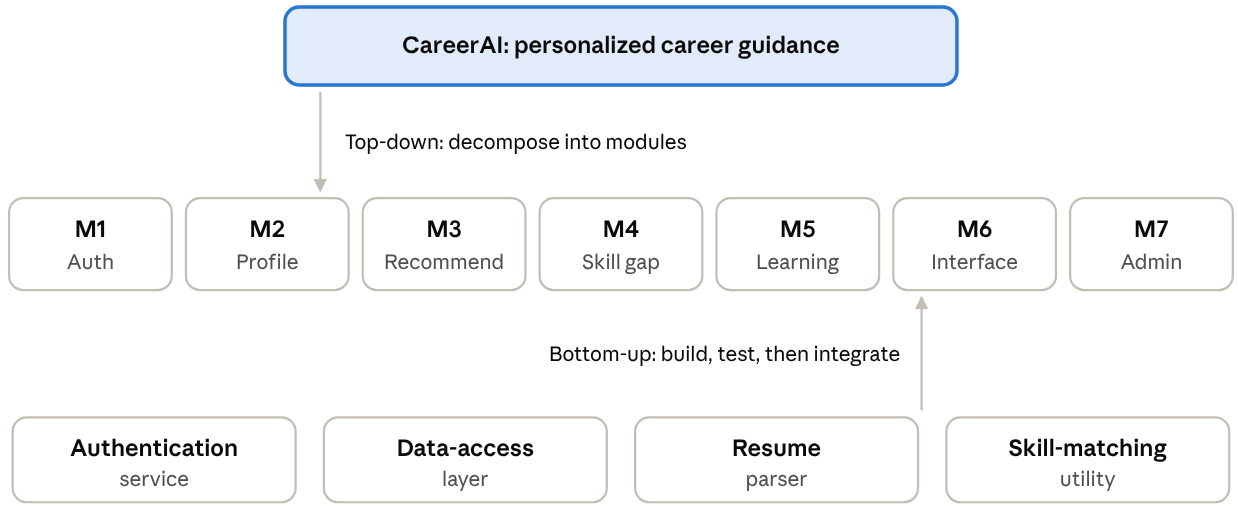
Selected strategy: **Hybrid** (Top-Down and Bottom-Up). Proposed.

Justification. CareerAI has a clear overall goal that decomposes naturally into functional modules, which suits top-down design. It also relies on reusable technical components, such as authentication, data access and resume parsing, that are best built and tested first and then integrated, which suits bottom-up design. A purely top-down plan would leave these shared components undefined until late, and a purely bottom-up plan would risk building parts that do not fit the recommendation workflow. The hybrid strategy keeps the structure driven by user needs while reusing proven building blocks across modules.

Evidence. Top-down: the system was first stated as one goal, personalized career guidance, then split into seven functional modules (M1 to M7), and each module into its main functions (for example, M3 into profile matching, rule evaluation, ranking and search/filter). Bottom-up: shared components (authentication service, data-access layer,

resume parser, skill-matching utility) are built and tested first, then assembled into M1, M2, M3, M4 and M5 and integrated behind the user interface (M6). The sketch below shows both directions.

Hybrid strategy: decompose from the top, build and integrate from the bottom



Reusable components are built first, then integrated into the modules behind the interface (M6).

decomposition and build-up sketch · 3 layers

The goal is decomposed top-down into seven modules, while shared components are built bottom-up and integrated into them.

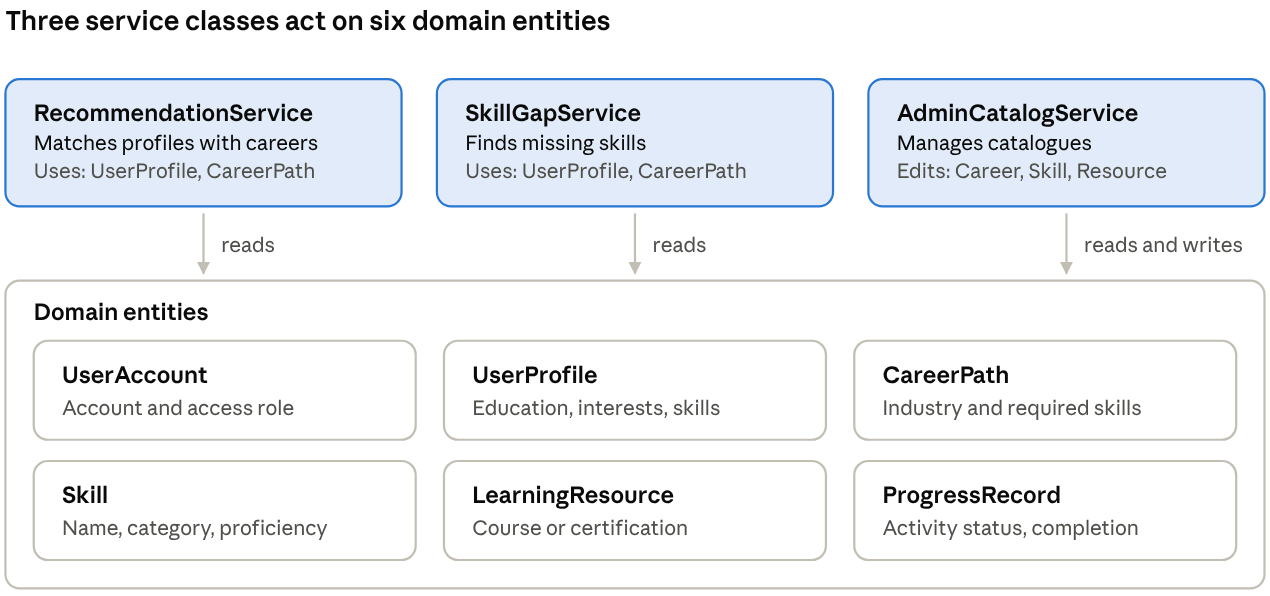
D. Design Approach

Selected approach: **Object-Oriented**. Proposed.

Justification. CareerAI is built around domain entities with their own data and behaviour: UserAccount, UserProfile, CareerPath, Skill, LearningResource and ProgressRecord. Operations such as career matching, skill-gap calculation and catalogue maintenance fit naturally as service classes (RecommendationService, SkillGapService, AdminCatalogService) that encapsulate logic behind small interfaces. Encapsulation supports low coupling, and new careers, skills or matching rules can be added through new classes or subclasses without rewriting existing code, which helps maintainability and scalability. A function-oriented design would spread the same data across many shared procedures and make this harder.

Class	Responsibility
UserAccount	Account information, authentication and access role.
UserProfile	Academic background, interests, skills and preferences.
CareerPath	Career description, industry, education requirements and required skills.
Skill	Skill name, category and proficiency information.
RecommendationService	Matches profiles with suitable career paths.
SkillGapService	Identifies missing skills for a selected career.
LearningResource	Course, certification or learning-material information.
ProgressRecord	Tracks learning activities and completion status.
AdminCatalogService	Manages career, skill and course catalogues.

Evidence. The structure chart below shows the entity classes and service classes and how they use one another.



class and service structure chart · 3 services, 6 entities

The services hold the matching, gap and catalogue logic, and the domain entities hold the data they operate on.

E. Coupling Analysis

The design aims for low coupling: modules exchange only the data they need through defined interfaces, and none reaches into another module's internal storage. Coupling levels below are the proposed target design.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M2 Profile Management → M3 Recommendation Engine	Normalized profile data (education, interests, skills, preferences) passed as a structured profile object.	Data coupling (low). Only the needed fields are exchanged.	Keep a stable, versioned profile interface so M3 never reads M2's storage; add new profile fields without changing M3's calls.
M3 Recommendation Engine → M7 Career and Skill Data	Career requirements and skill definitions read from the catalogue.	Data coupling (low), through a read-only catalogue interface.	Access catalogue data only via a query interface so M7's schema can change without altering M3; cache frequently used reads.
M4 Skill-Gap Analysis → M2 and M7	User skills (M2) and required skills for the selected career (M7), returning a list of gaps.	Data coupling (low). Two parameters in, one result out.	Pass skills as plain lists of Skill objects and keep the comparison inside M4; avoid sharing profile internals.
M5 Learning Resources → M4 Skill-Gap Analysis	Identified skill gaps used to select resources from the course catalogue.	Data coupling (low).	Receive a gap list, not M4's internal results; keep matching resources to skills inside M5.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M6 User Interface → services (M3, M4, M5)	User selections and feedback sent in; recommendations, gaps, resources and progress returned.	Data coupling (low) via service interfaces. Stamp coupling is possible if whole profile objects are passed.	Pass only the fields each view needs through small request/response objects; keep business logic out of the interface.
M1 Authentication → M2, M7	Authenticated identity and role used to authorize profile access and admin actions.	Data coupling (low) if only identity and role are shared.	Share a session token or identity object only; M2 and M7 never read credential data.

No common coupling (shared global data) or content coupling (one module modifying another's internals) is intended.

F. Cohesion Analysis

Each module is designed around one main responsibility so its internal tasks belong together. Levels below are the proposed target design.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
M4 Skill-Gap Analysis	Identify missing or underdeveloped skills for a selected career.	Functional cohesion (highest). Every task (read user skills, read career requirements, compare, report gaps) serves that single goal.	None. Keep resource suggestions out of this module.

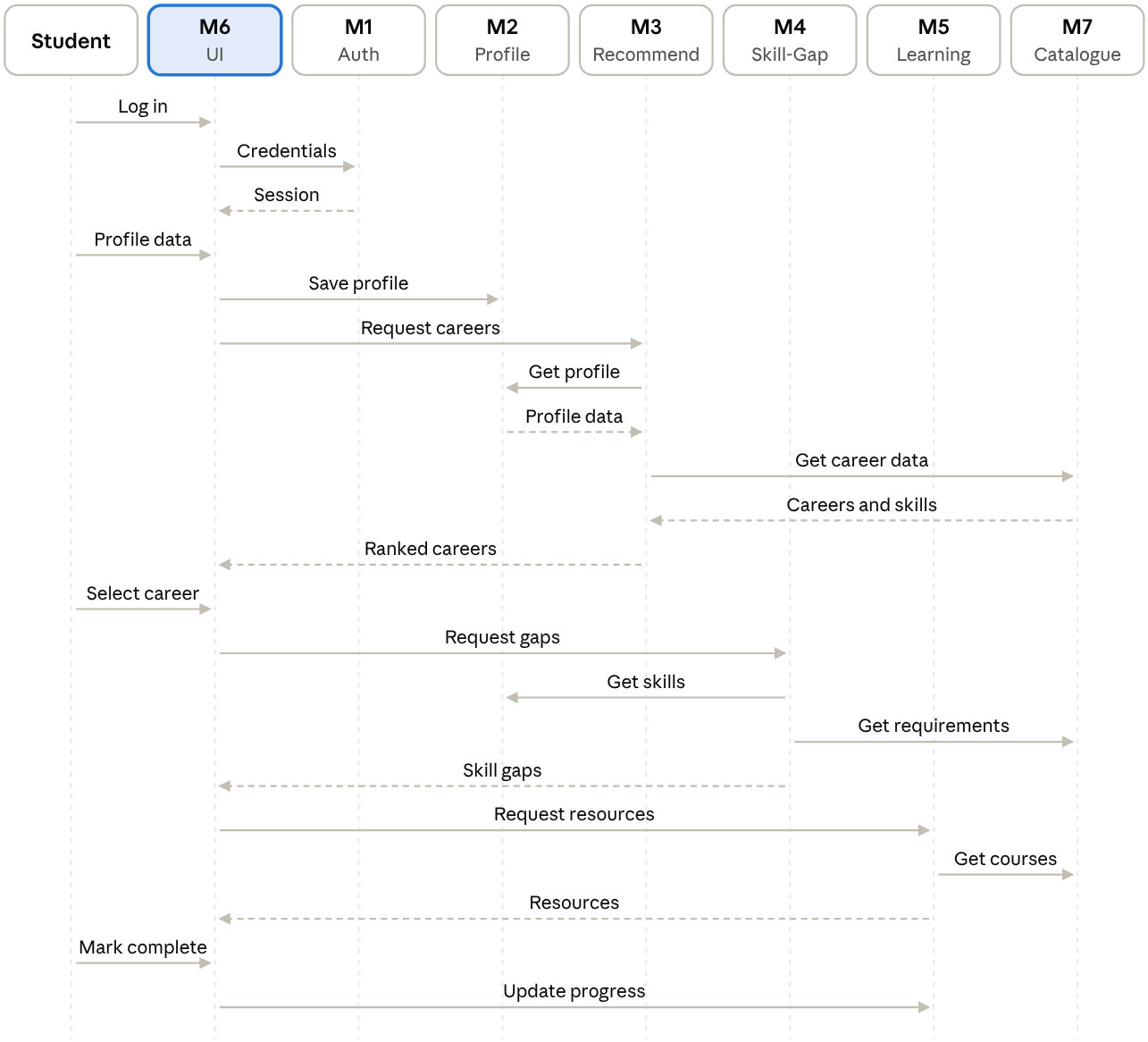
Module	Main responsibility	Cohesion type / level	Improvement, if needed
M5 Learning Resources and Progress Tracking	Recommend resources for skill gaps and track learning status.	Sequential/communicational cohesion (medium to high): tracking works on the same learning records that recommendations produce.	If it grows, split into a Learning Recommendation component and a Progress Tracking component behind separate interfaces.
M3 Career Recommendation Engine	Match profiles to careers and rank them.	Functional cohesion. Matching, rule evaluation (the four business rules), ranking and search/filter all support producing recommendations.	Keep filtering and comparison as helper components so the matching logic stays focused.
M2 User Profile Management	Create, store and update user profile data.	Communicational cohesion (high): tasks operate on the same profile data. Optional resume parsing is a separate concern.	Isolate resume skill extraction in its own reusable component so profile management stays focused on profile data.
M7 Administration and Career Data Management	Maintain careers, skills and courses; summarize usage.	Communicational cohesion (medium to high): all tasks act on catalogue and usage data.	Separate usage reporting from catalogue maintenance if reports grow beyond simple summaries.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
M6 User Interface and Feedback	Present results and capture user input and feedback.	Logical/communicational cohesion (medium): groups several views, search, notifications and feedback.	Split into view components per feature (dashboard, search, notifications) and keep all business logic in the services.

G. Module Interaction and Design Quality

The sequence diagram follows the example flow from the requirements: a student logs in, saves a profile, receives ranked careers, selects one, gets skill gaps and learning resources, then updates progress. It is a proposed interaction design, not a confirmed implementation.

Proposed flow: the interface coordinates and services do not call each other



module interaction sequence · 8 participants, 21 messages

Design quality. The interface (M6) coordinates the services, so M3, M4 and M5 never call one another and each stays replaceable. Services read profile and catalogue data only through the M2 and M7 interfaces, and only identity and role from M1 are shared across modules. Solid arrows are requests and dashed arrows are returned results; return arrows for the look-ups from M4 to M2 and M7 and from M5 to M7 are omitted for clarity.